Functions, Recursion and Lists

Concepts of Programming Languages
Lecture 2

CAS CS 320

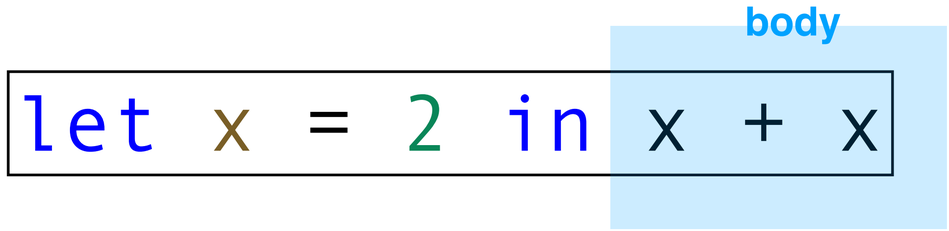
Outline

- Cover the **basic expressions** we need to start programming in OCaml, look at some examples
- Discuss **lists** so we can write actually interesting programs
- Talk about **tail recursion** and how that affects performance of OCaml programs

Basic Expressions

- Literals (e.g., 0, ‘a’, “abc”, true, -5.2)
- **Let-expressions (local variables)**
- If-expressions
- Functions
- Applications

Local Variables



We can define local variables in OCaml

This is useful for writing better abstractions

Note that it reads like a sentence: *let x stand for 2 in the expression x + x*

Multiple Local Variables

```python
def sum_of_squares(x, y):
  x_squared = x * x
  y_squared = y * y
  return x_squared + y_squared
```

Python

```
let sum_of_squares x y =
  let x_squared = x * x in
  let y_squared = y * y in
  x_squared + y_squared
```

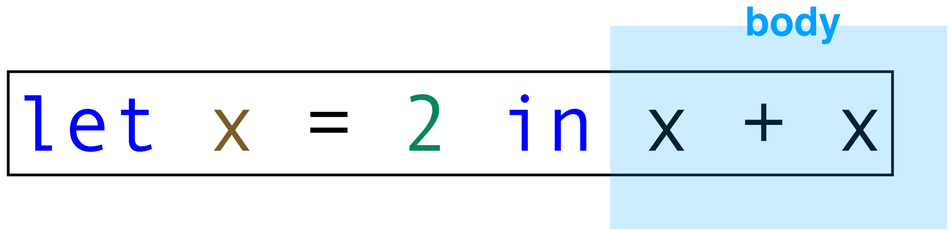
OCaml

It's very easy to use multiple local variables, we just *nest* local variables

(If it helps, think of **in** *as a semicolon ;)*

IMPORTANT: `let x = e1 in e2` is an *expression* so it can be the body of a let expression

Local Variables (Informal)



syntax: let VARIABLE = EXPRESSION in BODY

typing: the type is the same as that of BODY *given BODY is well-typed after substituting the VARIABLE in BODY*

semantics: the is the same as the value of BODY *after* ***substituting*** *the VARIABLE in BODY*

A Note on Substitution

Formally, we write $[v/x]e$ to mean "substitute v for x in e", e.g., $[3/x](x+x)$ is the same as $3+3$

Intuitively, substitution is simple: **replace the variable**

Turns out, this is **very hard** to do correctly, *it's subtle* and a source of a lot of mistakes in PL implementations

Basic Expressions

- Literals
- Let-expressions (local variables)
- **If-expressions**
- Functions
- Applications

If-Expressions

```
let abs x = if x > 0 then x else -x
```

OCaml has expressions for conditional reasoning

Note: The **else** case is *required* and the **then** and **else** cases must be the *same type* (why?)

If-Expressions

```
let foo x =
  if x < 0 then
    "negative"
  else if x = 0 then
    "zero"
  else
    "positive"
```

Answer: Remember, all we have is expressions. So every if-expression must have a value and a type (and therefore, an **else** case of the same type)

We can do **else if** just by nesting if-expressions! (neat)

If-Expressions (Informal)

```
let abs x = if x > 0 then x else -x
```

Syntax: if CONDITION then TRUE-CASE else FALSE-CASE

Typing: CONDITION must be a Boolean and TRUE-CASE and FALSE-CASE must be the same type. The type is then the same as that of TRUE-CASE and FALSE-CASE

Semantics: If CONDITION holds, then we get the TRUE-CASE, otherwise we get the FALSE-CASE

Basic Expressions

- Literals
- Let-expressions (local variables)
- If-expressions
- **Functions**
- Applications

Functions

```
let f x y z = x + y + z
let f (x : int) (y : int) (z : int) : int = x + y + z
```

Note: You don’t need to give types to arguments, but it’s good practice! At least in the start

There are a couple ways of defining functions in OCaml

Note that let-expression can take arguments. ***How should we interpret this? If everything is an expression?***

Anonymous Functions

```
let f = fun x -> fun y -> fun z -> x + y + z
```

Answer: It must be that **functions are expressions as well**!

In OCaml, we can define *anonymous* functions, which are just **functions without names**

You should think of:

```
let f x y z = x + y + z
```

as shorthand for the above

Aside: Type Annotations

```
let rec fact (n : int) : int =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

OCaml has type inference which means we rarely have to *specify* the types of expression in our program

That said, you **should** include type annotations, especially at the beginning, because they're useful for *documentation* and for *code clarity*

Aside: Anonymous Functions in Python

```
lambda x: x + 1
```

Python

```
fun x -> x + 1
```

OCaml

$$\lambda x . x + 1$$

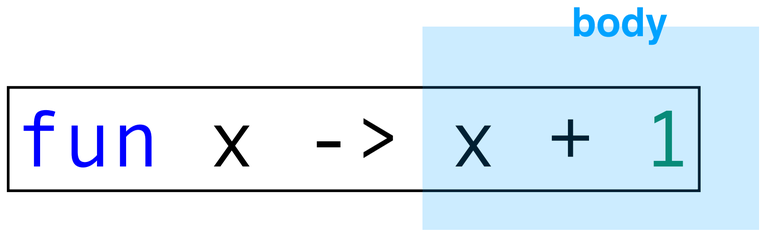
Math

There are also anonymous function in Python!

They're called *lambdas*, based on the **lambda calculus**, a mathematical formulation of a functional PL that dates back to the 1930s, invented by **Alonzo Church**

(You'll find a lot of functional ideas hidden in languages like Python)

Functions (Informal)



Syntax: fun VAR-NAME -> EXPR

Typing: the type of a function is **T1 -> T2** where T1 is the type of the input and T2 is the type of the output

Semantics: A function will evaluate to special ***function value*** (printed as <fun> by utop)

Important: Curried Functions

```
let f = fun x -> fun y -> fun z -> x + y + z
```

The only kind of function we have is *single argument*

This seems restrictive, but ultimately it doesn't affect us at all

We can *simulate* multi-argument functions with nested functions. This is called **Currying** after Haskell Curry

Important: Curried Functions

```
let f = fun x -> fun y -> fun z -> x + y + z
```

We should think of the above function as something which takes an input and returns **another function**

In other words, we *partially apply* the function

Basic Expressions

- Literals
- Let-expressions (local variables)
- If-expressions
- Functions
- **Applications**

Application

```
(fun x -> fun y -> x + y + 1) 3 2
```

Application is done by *juxtaposition* which means we put the arguments next to the function

Application is *left-associative*, which means we pass arguments from left to right

Application (Informally)

```
(fun x -> fun y -> x + y + 1) 3 2
```

Syntax: FUNCTION-EXPR ARG-EXPR

Typing: If FUNCTION-EXPR is of type T1 -> T2, and ARG-EXPR is of type T1, then the type is T2

Semantics: Substitute the value of ARG-EXPR into the body of FUNCTION-EXPR and evaluate that

Application (Example)

```
(fun x ->  fun y -> x + y  + 1)  3  2
```

is the same as

```
(fun x -> (fun y -> x + y  + 1)) 3  2
```

is the same as

```
((fun x -> (fun y -> x + y  + 1)) 3) 2
```

evaluates to

```
(fun y -> 3 + y  + 1)    2
```

evaluates to

```
3 + 2  + 1
```

is the same as

```
(3 + 2) + 1
```

evaluates to

```
5 + 1
```

evaluates to

```
6
```

Lists

What is a list?

```
let _ = 1 :: 2 :: 3 :: []
let _ = 1 :: (2 :: (3 :: []))
let _ = [1; 2; 3]
```

A list is an ordered ***variable-length homogeneous*** collection of data

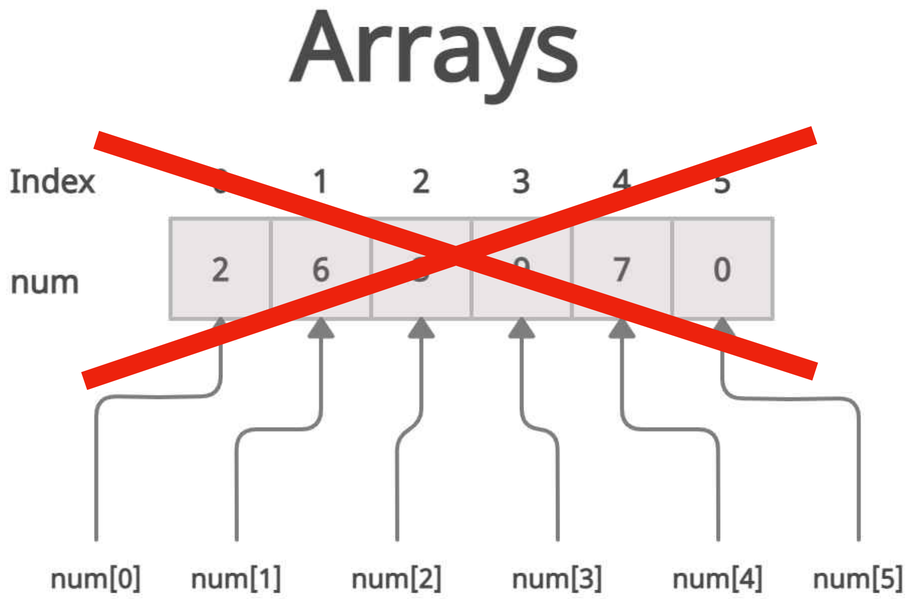
Many important operations on data can be represented as operations on lists (e.g., updating all users in a database)

What is a list not?

A list is *not* an array. We don't have constant-time indexing

A list is *not* mutable. **No data structures in FP are mutable**

(You should think of a list structurally as more like a linked list, sort of)



https://usemynotes.com/what-are-arrays-in-java/

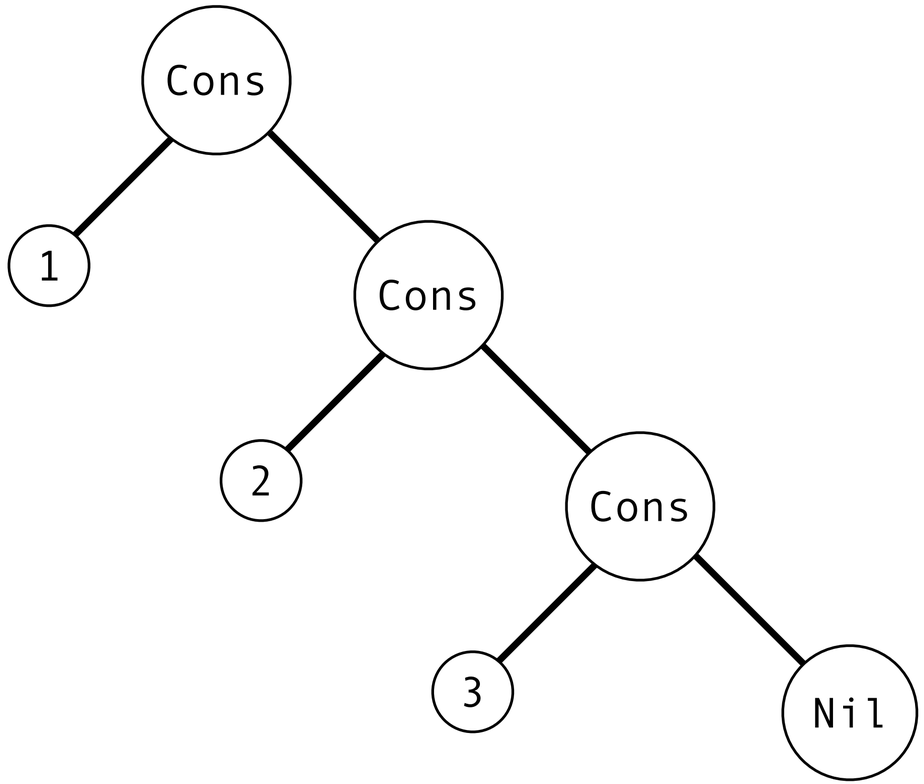
The Picture

We can think of the list

```
1 :: 2 :: 3 :: []
```

as a leaning tree with data a leaves

(this will generalize to other *algebraic* data types)



Constructing Lists (Informally)

```
let _ = 1 :: 2 :: 3 :: []
let _ = 1 :: (2 :: (3 :: []))
let _ = [1; 2; 3]
```

`[]` stands for the empty list (a.k.a. **nil**), the list with no elements

x :: xs stands for the list xs with x prepended to it. The symbol :: is pronounced "cons" and is a *right associative* operator

[x1; x2;...; xn] is a list literal. It's shorthand for a list of a known length

Example

*Construct a function **generate** which, given integers **n**, returns a list consisting of the first **n** positive integers*

Lists Semantics (Informally)

[2 + 3; 4 * 12; 2 - 1] ⇓ [5; 48; 1]

We evaluate the list [e_1; e_2; ... ; e_k] by evaluating each element of the list (from right to left)

Destructing Lists

```
match l with
| [] -> (* something *)
| x :: xs -> (* something else *)
| ... (* other patterns??? *)
```

As with any type in OCaml, we can use **pattern matching** to destruct lists

With pattern matching, we describe the value we want based on the *shape* of the list we're matching on

Example

Implement the function ***double*** *where* ***double l*** *is the same as the list* ***l*** *but with every element doubled*

Lists are Immutable

```
val remove_all_negatives : int list -> int list
```

All data structures in FP are immutable

If we want to "update" a list, we have to produce an *entirely new* list

In reality the data is not literally duplicated, there are optimizations which allow for shared data

Practice Problem

Implement the function

remove_all_negatives : int list -> int list

where ***remove_all_negative l*** *is the same as the list* ***l*** *but with all negative numbers removed*

Deep Pattern Matching

```
match <expr> with
| [] -> <expr>
| [h1; h2] -> <expr>
| h1::h2::t -> <expr>
| h::t -> <expr>
| ......
```

Pattern matching is very general. We can match on more complex patterns than just empty and nonempty

Example

Implement the function

delete_every_other : int list -> int list

such that ***delete_every_other l*** *is the first, third, fifth,..., and so on elements of* ***l***

Tail Recursion

demo

(even the wrong way)

Tail Recursion

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

not tail recursive

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

tail recursive

A recursive function is **tail recursive** if it does not perform any computations on the result of a recursive call

Why do we care?

Recursive functions are *expensive* with respect to the call-stack

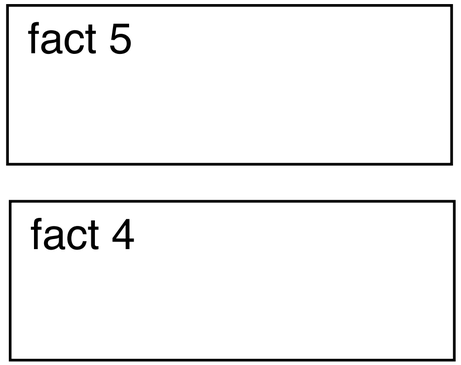
Tail-call elimination is an optimization implemented by OCaml's compiler which *reuses* stack frames

Tail-recursive functions "behave iteratively"

The Picture


```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

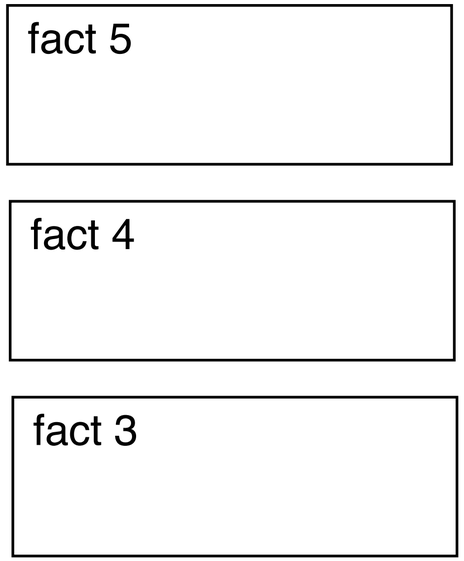
The Picture




```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

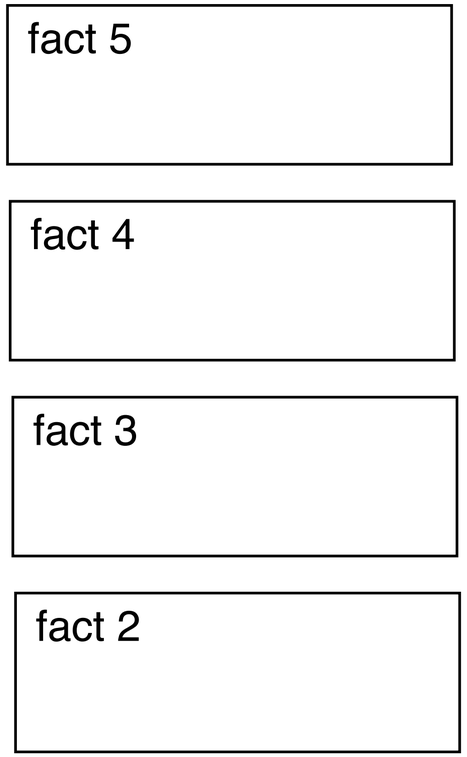
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



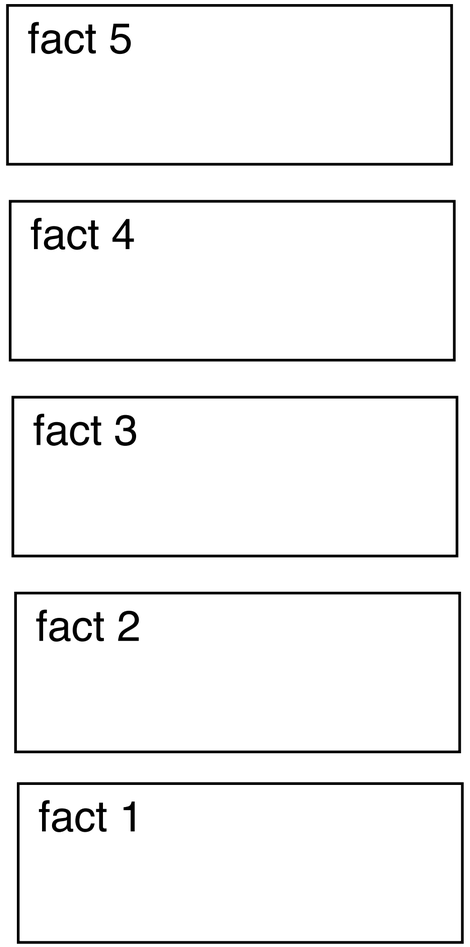
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



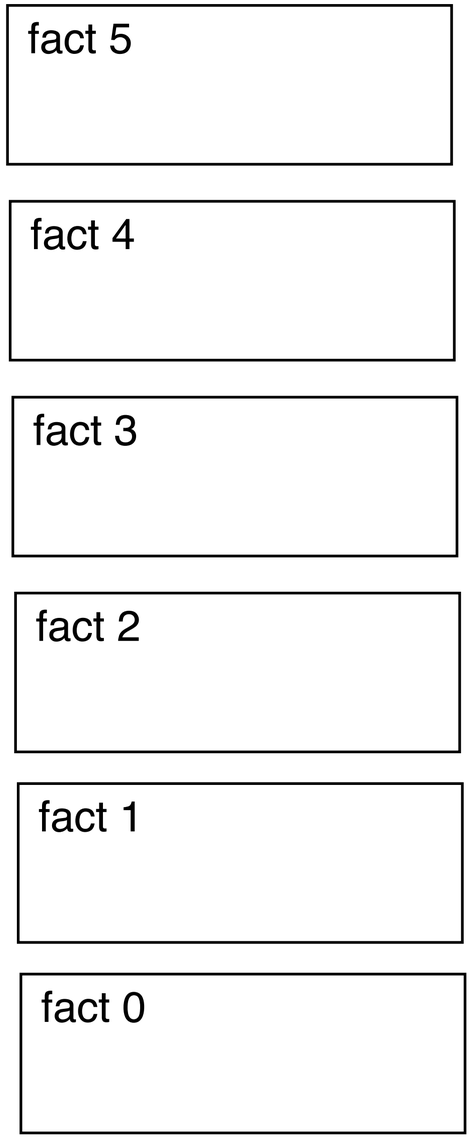
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



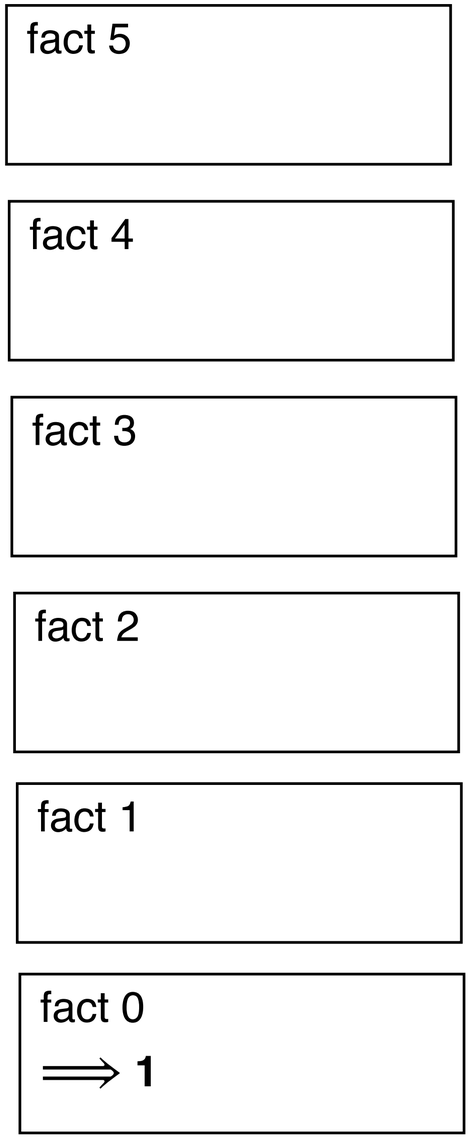
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



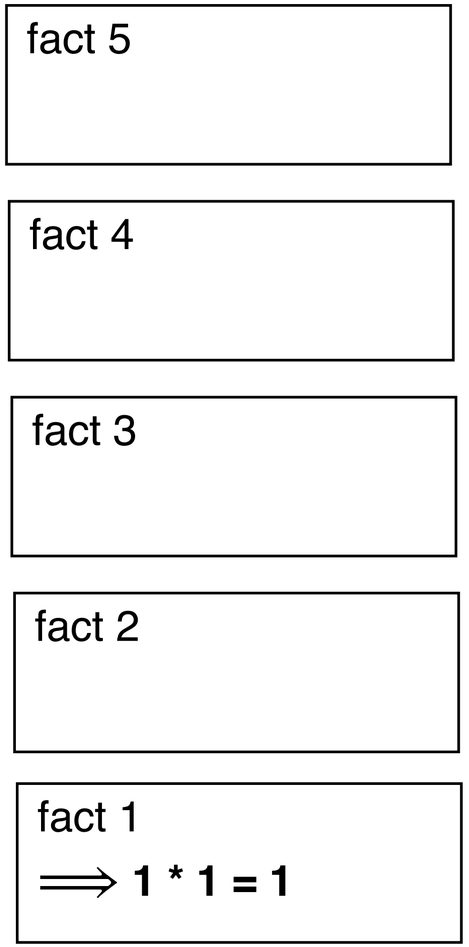
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



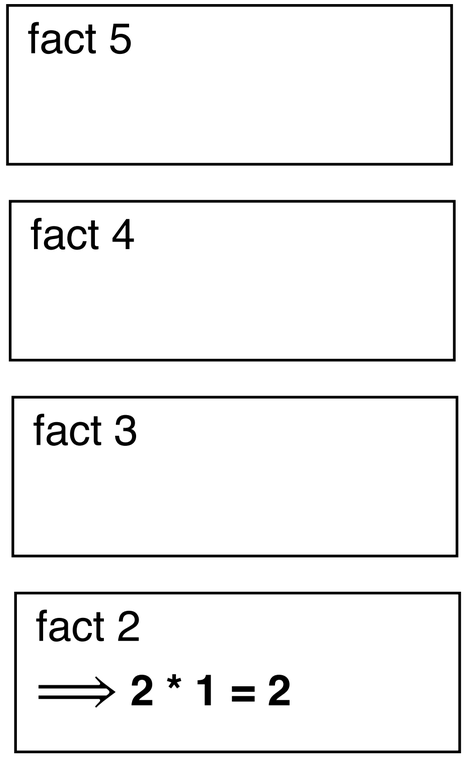
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



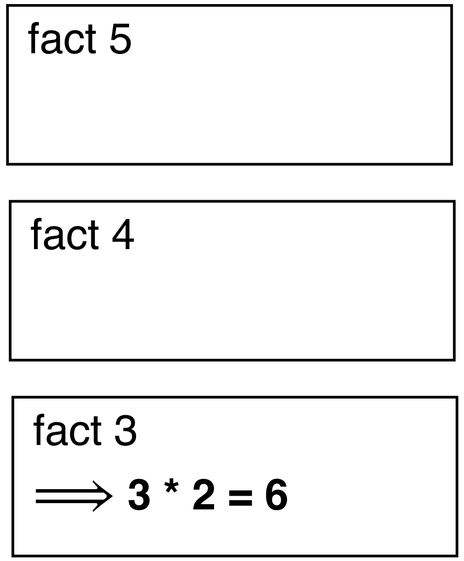
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



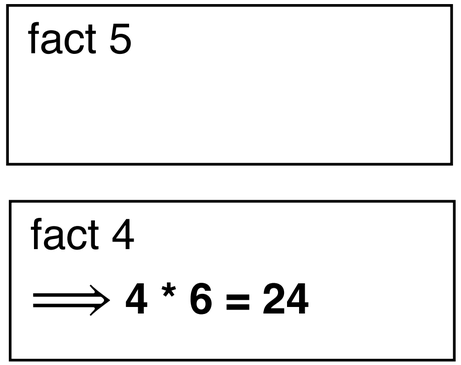
The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```



The Picture

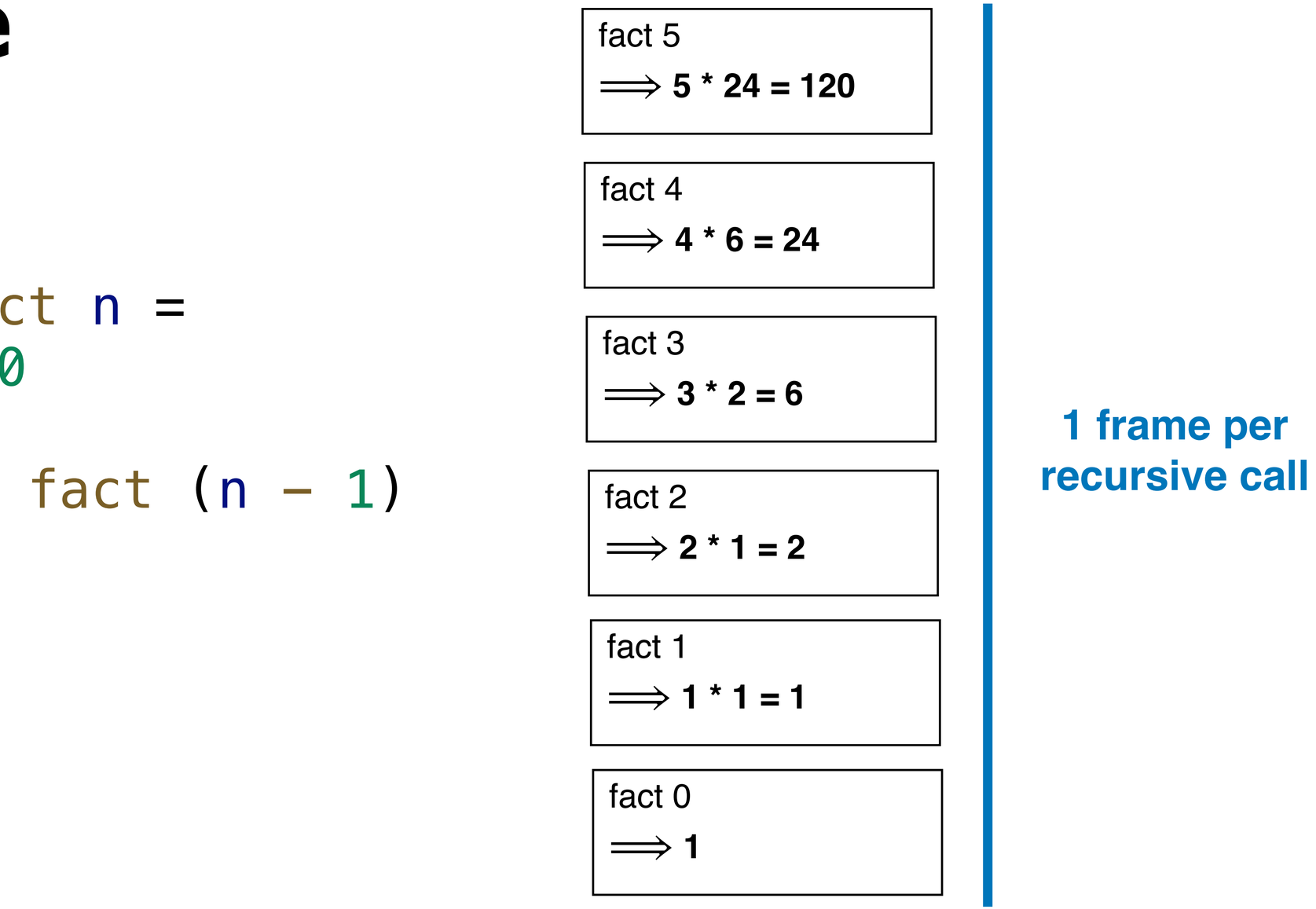
fact 5

$\Longrightarrow$ **5 * 24 = 120**

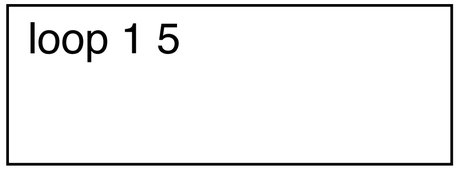
```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

The Picture

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

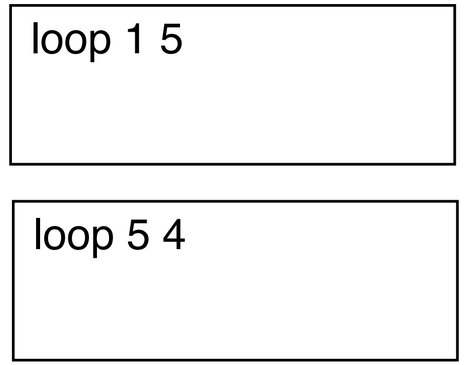


The Picture




```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

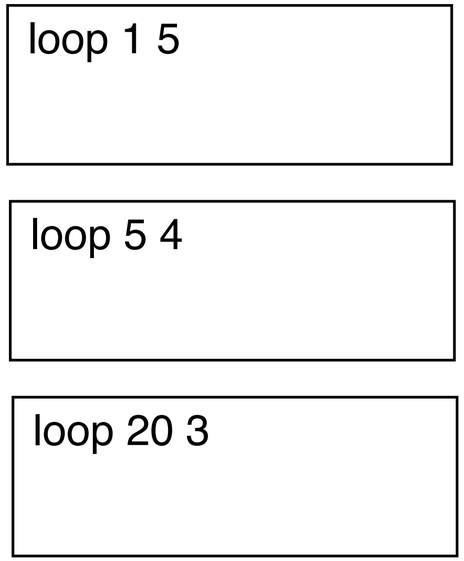
The Picture




```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

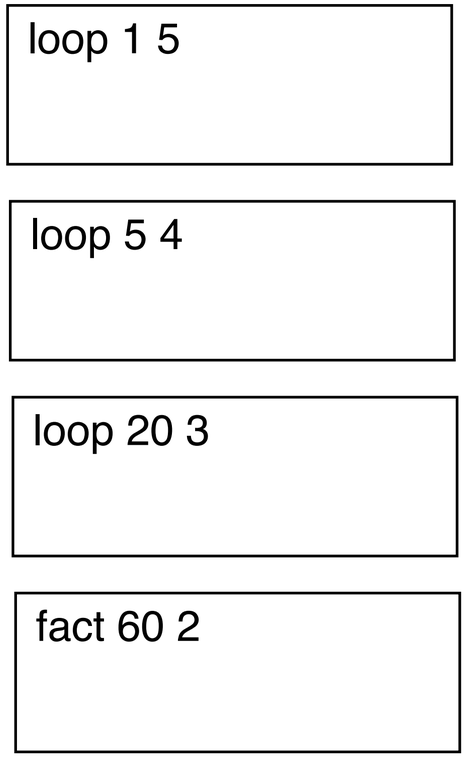
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



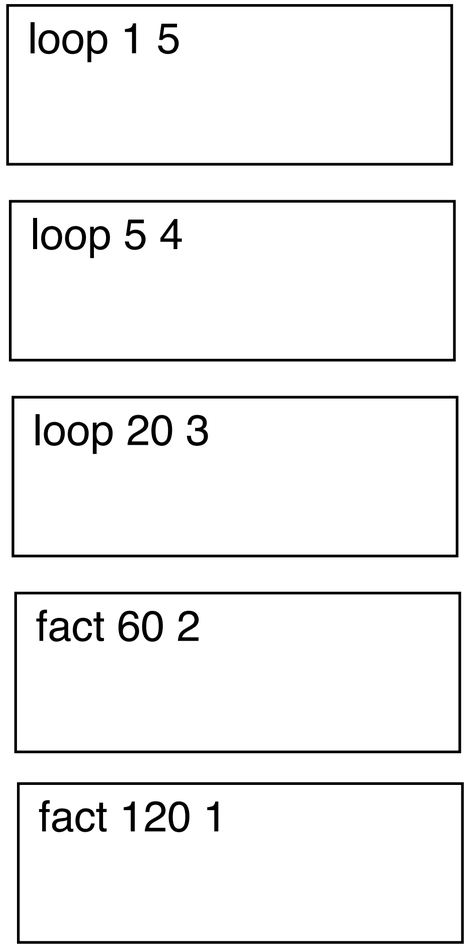
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



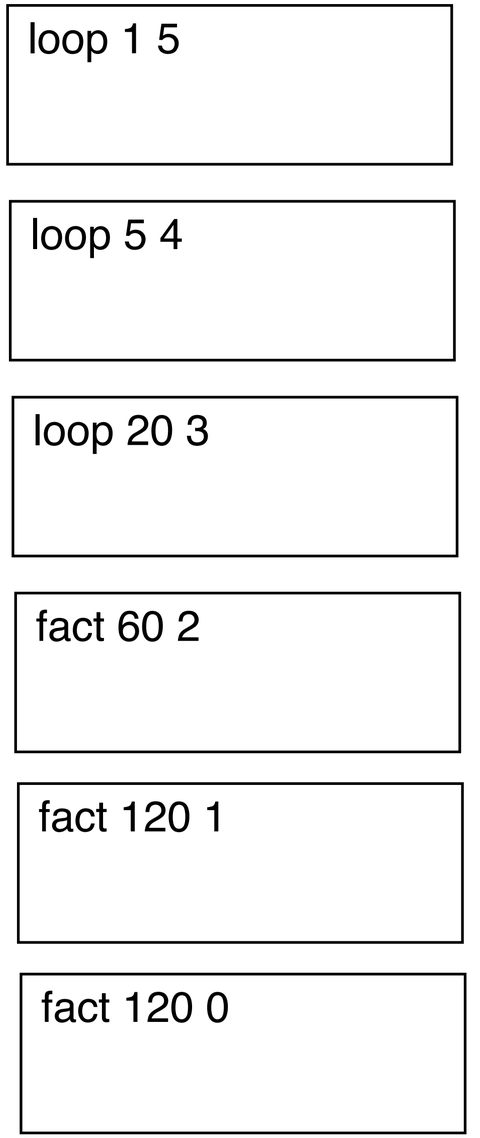
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



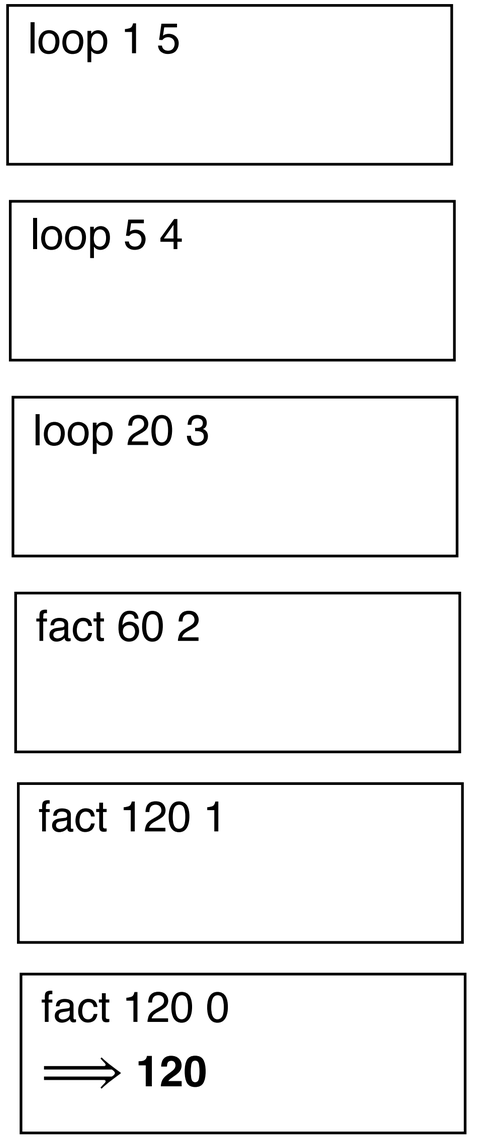
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



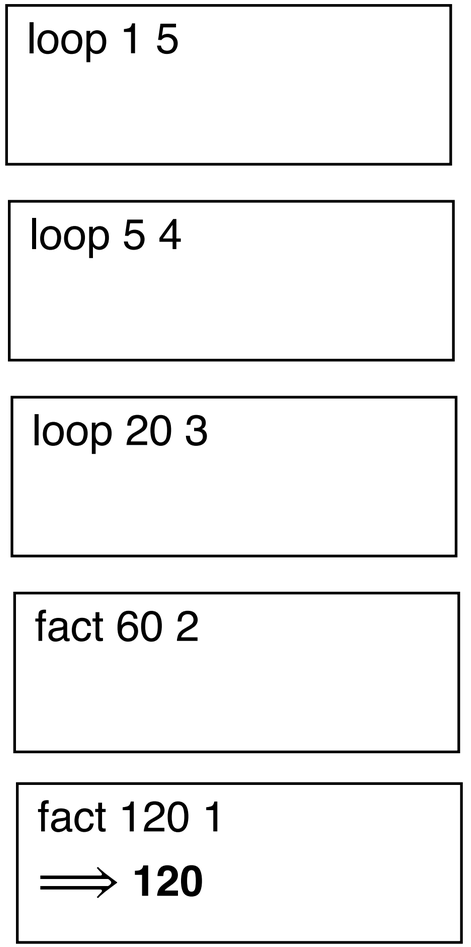
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



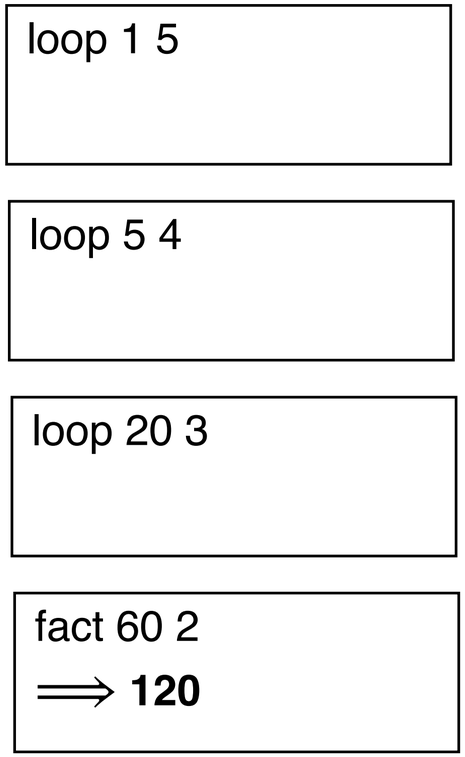
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



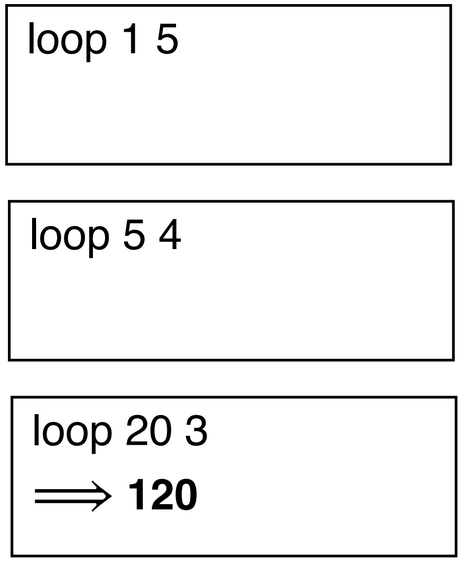
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



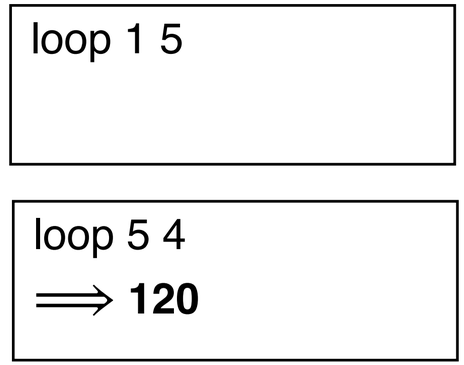
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



The Picture

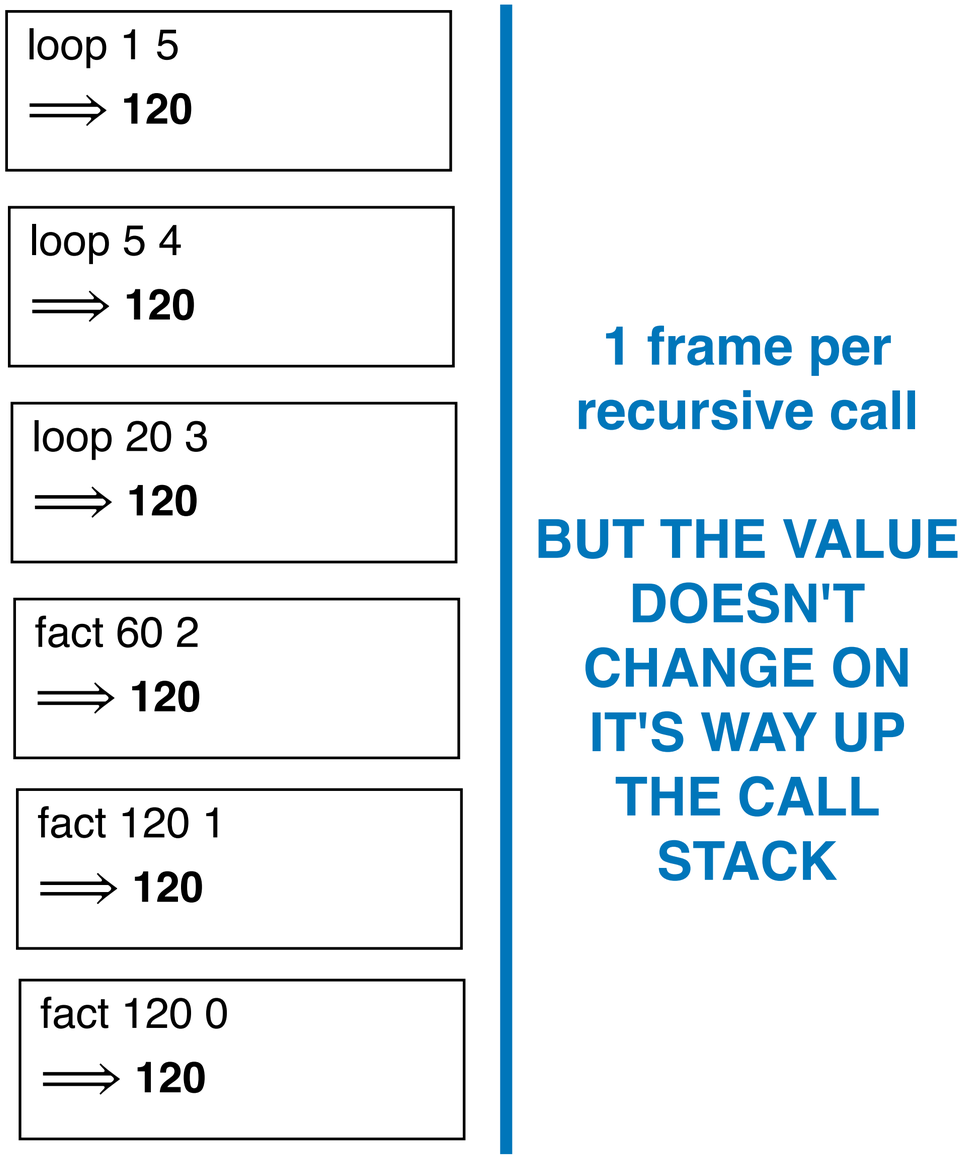
loop 1 5

$\Longrightarrow$ **120**

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

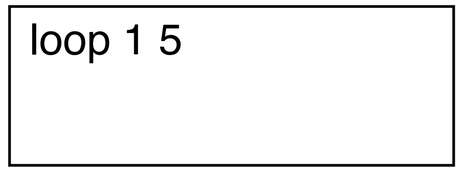
The Picture

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



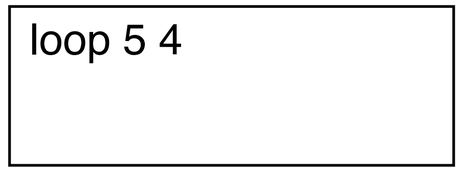
The Picture (Optimized)

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



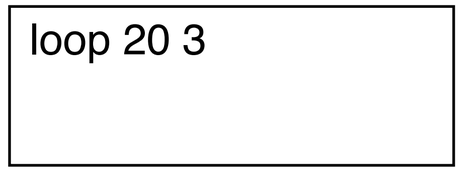
The Picture (Optimized)

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



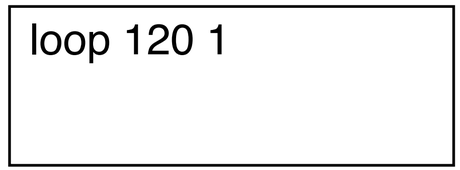
The Picture (Optimized)

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



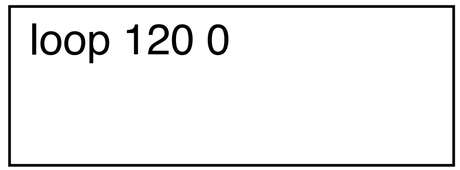
The Picture (Optimized)

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



The Picture (Optimized)

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



The Picture (Optimized)

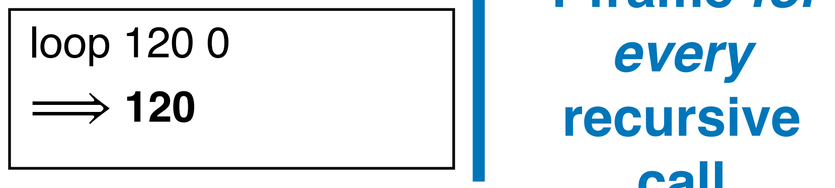
```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

loop 120 0

$\Longrightarrow$ **120**

The Picture (Optimized)

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```



Tail Position

```
let rec fact n =
  if n <= 0
  then 1
  else n * fact (n - 1)
```

computation after the recursive call

not tail recursive

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

tail recursive

Tail-call optimizations apply to functions whose recursive calls are in **tail position**

Intuition: A call is in tail position if there is no computation *after* the recursive call

Aside: Tail Position More Formally

let rec f x1 x2 ... xk = e

A recursive call **f e1 e2 ... ek** * is in tail position in **e** if:

- it does not appear in **e**, or **e** is the recursive call itself
- e = **if e1 then e2 else e3** and the call does not appear in **e1** and it is in tail position in **e2** and **e3**
- e is a **match-expression** and the call is in tail position in every branch, and does not appear in the matched expression
- e = **let x = e1 in e2** and the call does not appear in the **e1** and it is in tail position in **e2**

* f cannot appear in e1 ... ek

Accumulators

```
let fact n =
  let rec loop acc n =
    if n <= 0
    then acc
    else loop (n * acc) (n - 1)
  in loop 1 n
```

Our accumulator pattern is almost always tail recursive

Code Example

Implement the function

reverse : 'a list -> 'a list

The implementation must be tail recursive

Summary

OCaml is a language of **expressions,** everything is an expression

Lists are used to process collections of homogeneous data

We can use **tail-recursion** to make our implementations more memory efficient, but we have to be careful when working with lists